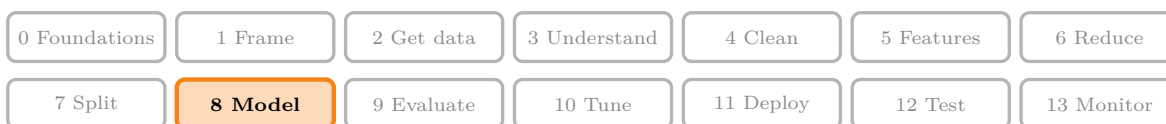


Self-Attention, Geometrically

Contents

1. Overview	2
2. Prerequisites	3
3. Words as arrows	4
4. Step 1: three projections of every word	4
5. Step 2: compare the query with every key	5
6. Step 3: a weighted sum of the value vectors	6
7. What the picture means	7
7.1 The output sits between the value vectors	7
7.2 The same word in another sentence	8
7.3 Adding the change back	8
8. Summary	11
9. Sources	11
10. Key terms	11

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Attention mechanism ([Note 1069](#)).
- **Leads to:** Positional encoding ([Note 1078](#)); Masked self-attention ([Note 1081](#)).
- **Compare with:** Cross-attention ([Note 1082](#)).

1. Overview

Key point: Drawn as arrows, self-attention does three things to a word’s vector. Self-attention projects the embedding into a query, a key and a value vector; it measures how well the word’s query points along every key; and it replaces the word by a weighted average of the value vectors. The new vector lies between the value vectors, pulled towards the words that got the most weight, so the same word lands in a different place in a different sentence.

The [self-attention step by step Note](#) and the [scaled dot-product attention Note](#) gave the formula for self-attention:

$$Y = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

This Note draws what the formula does to the vectors. Real embeddings have hundreds of numbers, which we cannot draw. Here every embedding has just 2 numbers, so each vector is an arrow on a page. Figure 1 runs the whole computation for the word “bank” in the sentence “money bank”.

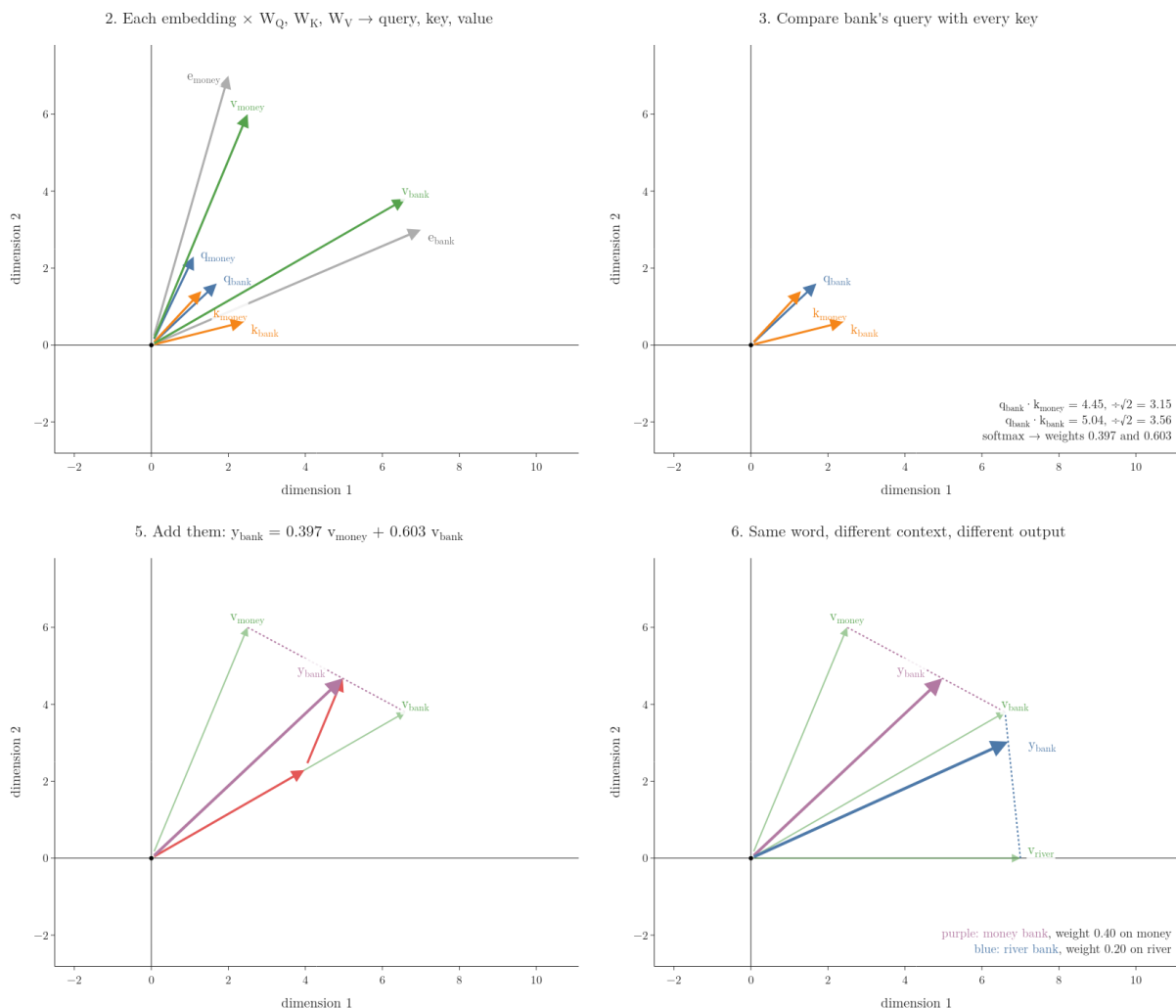


Figure 1: Self-attention on “money bank” in 2 dimensions: embeddings, their query (blue), key (orange) and value (green) vectors, the scores of bank’s query, the value vectors scaled by the weights, their sum y_{bank} , and finally y_{bank} in “river bank” for comparison

All the numbers in this Note are hypothetical, chosen by hand so that the arrows are easy to see. In a real transformer, W_Q , W_K and W_V are learned by training. The Notebook computes every number and checks the geometric claims.

2. Prerequisites

- [Self-attention step by step Note](#): query, key and value vectors, and the weighted sum of values.
- [Scaled dot-product attention Note](#): dividing the scores by $\sqrt{d_k}$.
- [What is self-attention Note](#): contextual embeddings, and the two meanings of “bank”.
- [Linear transformations and matrices Note](#): multiplying a vector by a matrix moves the vector.

- [Dot product and cosine similarity Note](#), section 5: vectors pointing the same way have a large dot product.

3. Words as arrows

Key point: An embedding is a list of numbers, so it is a point, or an arrow from the origin, in a space with one axis per number. Words used in similar ways sit close together.

A word embedding (the [RNN sentiment analysis Note](#), section 6) gives every word a vector. With 2 numbers per word, the first number is the position along one axis and the second along the other. Section 7.3 of the same Note plots learned 2-number embeddings of IMDB words: positive words and negative words end up on opposite sides. Embeddings with hundreds of numbers are often drawn the same way after a dimensionality reduction such as PCA, and words with similar meanings form clusters.

Our sentence is “money bank”. The two embeddings are

$$e_{\text{money}} = (2, 7), \quad e_{\text{bank}} = (7, 3)$$

They point in quite different directions (grey arrows in Figure 1, first frame).

4. Step 1: three projections of every word

Key point: Multiplying an embedding by W_Q , W_K and W_V moves it to three new places: its query, key and value vectors. One arrow becomes three.

Multiplying a vector by a matrix is a linear transformation: it moves the vector somewhere else (the [linear transformations Note](#)). Self-attention applies three such transformations to each embedding. Our 2×2 matrices are

$$W_Q = \begin{pmatrix} 0.2 & 0.1 \\ 0.1 & 0.3 \end{pmatrix}, \quad W_K = \begin{pmatrix} 0.3 & 0 \\ 0.1 & 0.2 \end{pmatrix}, \quad W_V = \begin{pmatrix} 0.9 & 0.2 \\ 0.1 & 0.8 \end{pmatrix}$$

1. **In words:** each projection is the embedding (a row vector) times a matrix.
2. **Formula:** $q = e W_Q$, $k = e W_K$, $v = e W_V$.
3. **Example:** for “money”,

$$q_{\text{money}} = (2 \times 0.2 + 7 \times 0.1, \quad 2 \times 0.1 + 7 \times 0.3) = (1.1, 2.3)$$

All six vectors (Figure 1, second frame, and Figure 2):

Word	Embedding e	Query q	Key k	Value v
money	(2, 7)	(1.1, 2.3)	(1.3, 1.4)	(2.5, 6.0)
bank	(7, 3)	(1.7, 1.6)	(2.4, 0.6)	(6.6, 3.8)

Two embeddings have become six vectors. The embeddings themselves are not used again inside the attention computation.

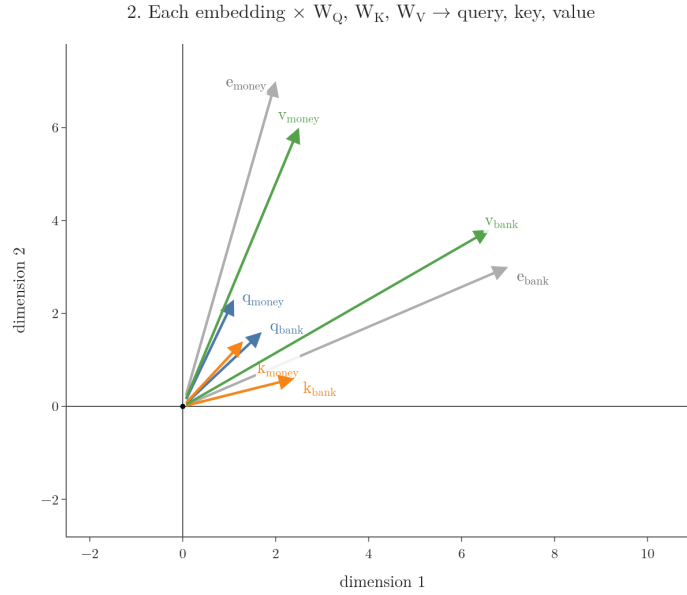


Figure 2: Step 1 as a still: the grey embeddings and their query (blue), key (orange) and value (green) vectors

5. Step 2: compare the query with every key

Key point: The dot product of bank’s query with each key measures how closely the two arrows point the same way. Scaling and the softmax turn these scores into weights that sum to 1.

We follow the word “bank”; “money” goes through exactly the same steps. Its new vector depends on how similar “bank” is to every word of the sentence, itself included. The similarity of two arrows is their dot product: large when they point the same way, small when the angle between them is wide (the [dot product Note](#), section 5).

1. **In words:** take the dot product of bank’s query with every key, divide by $\sqrt{d_k}$, then apply the softmax.
2. **Formula:**

$$w_{bank,j} = \text{softmax}_j \left(\frac{q_{bank} \cdot k_j}{\sqrt{2}} \right), \quad j \in \{money, bank\}$$

3. **Example:**

$$q_{bank} \cdot k_{money} = 1.7 \times 1.3 + 1.6 \times 1.4 = 4.45, \quad q_{bank} \cdot k_{bank} = 1.7 \times 2.4 + 1.6 \times 0.6 = 5.04$$

Divided by $\sqrt{2} = 1.414$: 3.15 and 3.56. The softmax gives

$$w_{bank,money} = 0.397, \quad w_{bank,bank} = 0.603$$

In Figure 1 (third frame), q_{bank} points between the two keys, a little closer to k_{money} in direction but with k_{bank} longer, so k_{bank} wins the larger weight (Figure 3). Once the weights are known, the queries and keys have done their job; only the value vectors are needed from here on.

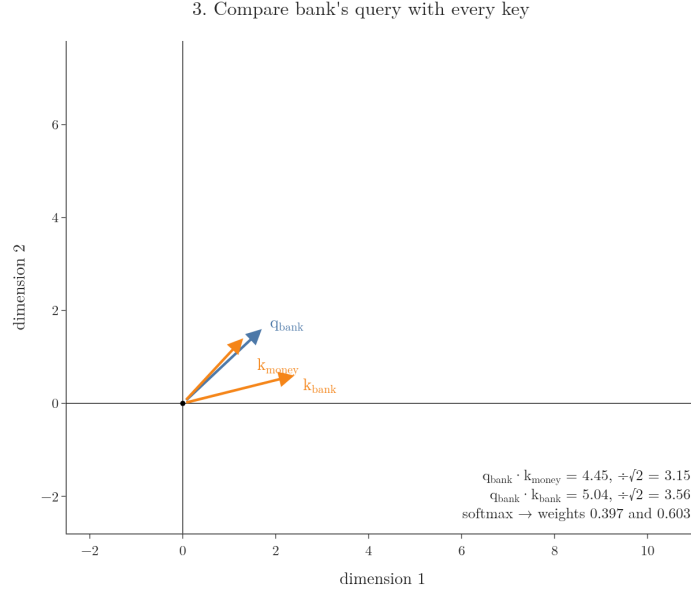


Figure 3: Step 2 as a still: bank’s query (blue) against the two keys (orange), with the scores and weights

6. Step 3: a weighted sum of the value vectors

Key point: Each value vector is shrunk by its weight, and the shrunk arrows are added tip to tail. The result y_{bank} is the contextual embedding of “bank”.

1. **In words:** multiply each value vector by its weight (multiplying a vector by a number shrinks or stretches it without turning it), then add the results.
2. **Formula:**

$$y_{bank} = w_{bank,money} v_{money} + w_{bank,bank} v_{bank}$$

3. **Example:**

$$0.397 \times (2.5, 6.0) = (0.99, 2.38), \quad 0.603 \times (6.6, 3.8) = (3.98, 2.29)$$

$$y_{bank} = (0.99 + 3.98, 2.38 + 2.29) = (4.97, 4.67)$$

Adding two arrows means placing the second at the tip of the first (the triangle rule), or completing the parallelogram they span; both give the same arrow. Figure 1 (fourth and fifth frames) and Figure 4 show the two shrunk value vectors in red and their sum y_{bank} in purple.

5. Add them: $y_{\text{bank}} = 0.397 v_{\text{money}} + 0.603 v_{\text{bank}}$

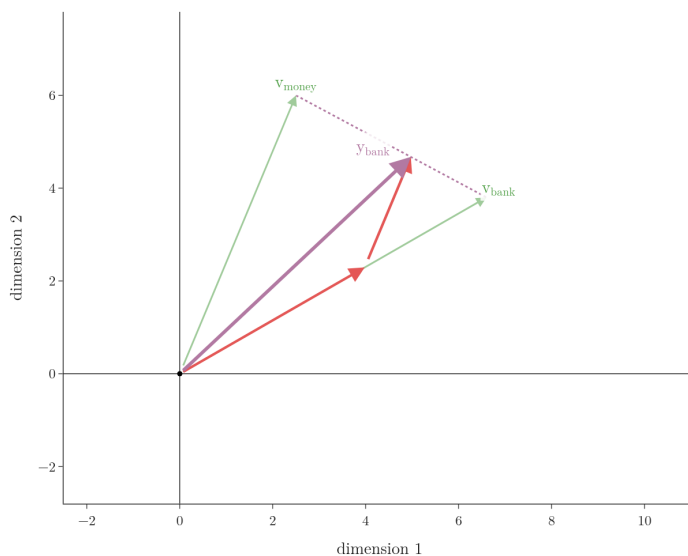


Figure 4: Step 3 as a still: the shrunk value vectors (red) placed tip to tail; their sum is y_{bank} (purple), on the dotted line between the two value vectors

7. What the picture means

Key point: y_{bank} always lies on the line between v_{bank} and v_{money} , a fraction $w_{\text{bank,money}}$ of the way towards v_{money} . Another context word pulls it somewhere else.

7.1 The output sits between the value vectors

Because the two weights are positive and add up to 1, the weighted sum can be rewritten:

$$y_{\text{bank}} = w v_{\text{money}} + (1 - w) v_{\text{bank}} = v_{\text{bank}} + w (v_{\text{money}} - v_{\text{bank}}), \quad w = w_{\text{bank,money}}$$

So y_{bank} starts at v_{bank} and moves a fraction $w = 0.397$ of the way along the straight line to v_{money} (the dotted line in Figure 1). Money pulls bank towards itself, like gravity: the more weight money gets, the stronger the pull. Bank pulls money too. Money's own weights are 0.61 on itself and 0.39 on bank, so $y_{\text{money}} = (4.10, 5.14)$ moves 39% of the way from v_{money} towards v_{bank} (Notebook).

Extra: With n words the same argument holds: the weights are positive and sum to 1, so each output is a weighted average of the n value vectors. A weighted average of points always lies inside the smallest region with straight edges that contains them (their convex hull): for 3 words, inside the triangle of their value vectors. Self-attention can only mix the value vectors of the sentence; it cannot produce a vector outside their range.

7.2 The same word in another sentence

Now replace “money” by “river”, with embedding $e_{river} = (8, -2)$, and keep everything else. The Notebook repeats the steps:

	“money bank”	“river bank”
Value of the other word	$v_{money} = (2.5, 6.0)$	$v_{river} = (7.0, 0.0)$
Weight of bank on the other word	0.397	0.202
y_{bank}	(4.97, 4.67)	(6.68, 3.03)
Distance moved from v_{bank}	1.85	0.77

The word “bank” has the same embedding and the same value vector in both sentences, yet its outputs are 2.37 apart (Figure 1, last frame). In “money bank” it is pulled up towards money; in “river bank” it is pulled down towards river. Self-attention gives a word a different vector in each context, which is what the [what is self-attention Note](#) asked for: a **contextual embedding**.

In a trained model the matrices, and therefore the weights and the directions of the pulls, come from the training data. Our hand-picked numbers only show the geometry.

7.3 Adding the change back

Key point: In a transformer, the attention output is not the word’s new vector on its own. It is a change, Δe , that is **added** to the word’s own vector: new vector = $e + \Delta e$. The addition is the residual connection of the transformer encoder.

Sections 6 and 7.1 treated y_{bank} as the new vector of “bank”. Inside a transformer one more step follows. The attention output is added to the vector that went in, $e_{bank} + y_{bank}$ (the [transformer encoder Note](#), section 5.2, calls this the residual connection; there a layer normalisation follows the addition). So the value vectors are best read as “what to add to the other word if this word is relevant to it”, and their weighted sum y_{bank} as the change Δe (Sanderson 2024, Ch 6). Section 7.1 still holds: the change itself lies between the value vectors. What moves the word is the change added on top of it.

1. **In words:** keep the word’s own arrow, and put the attention output at its tip.
2. **Formula:** $e_{bank}^{new} = e_{bank} + \Delta e$, $\Delta e = y_{bank} = w_{bank,money} v_{money} + w_{bank,bank} v_{bank}$
3. **Example:** in “money bank”, $e_{bank} + \Delta e = (7, 3) + (4.97, 4.67) = (11.97, 7.67)$. The arrow turns from 23 degrees to 33 degrees, towards “money”. In “river bank”, $(7, 3) + (6.68, 3.03) = (13.68, 6.03)$, at 24 degrees: almost no turn, only longer (Notebook).

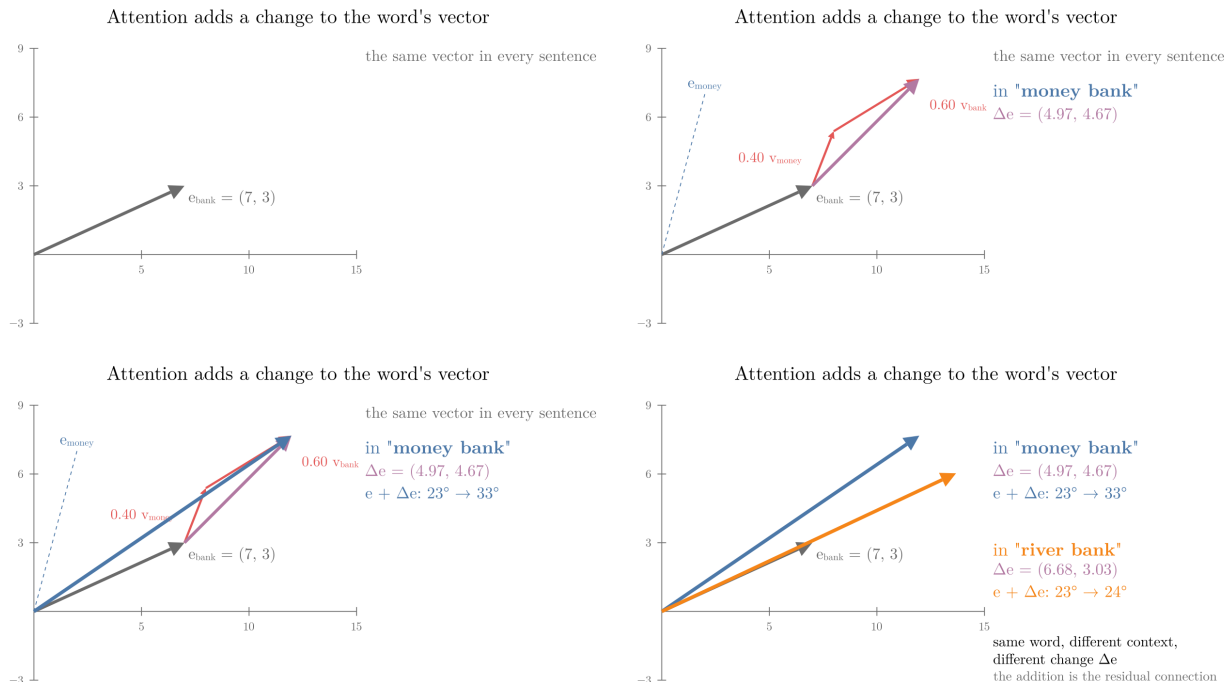


Figure 5: The attention output as a change added to the word’s vector. The weighted value vectors (red) are placed at the tip of e_{bank} (grey); their sum is Δe (purple); $e_{bank} + \Delta e$ is the new vector, blue for “money bank” and orange for “river bank”

Watch, in Figure 5, how the same grey arrow receives a different purple change in each sentence and so ends in a different place. With our hand-picked numbers the change is almost as long as the vector itself.

In a real trained model the change is mostly much smaller: a nudge. The Notebook runs GPT-2 small, a published trained transformer with 12 blocks and 768 numbers per token (Radford et al. 2019), and measures, for the last word “bank” of two real sentences, the length of each attention block’s change relative to the vector it is added to (Figure 6). In blocks 2 to 11 the change is 9 to 25 percent of the vector’s length, and the vector after the addition still points almost the same way as before (cosine at least 0.97). Only the first and the last block change the vector a lot.

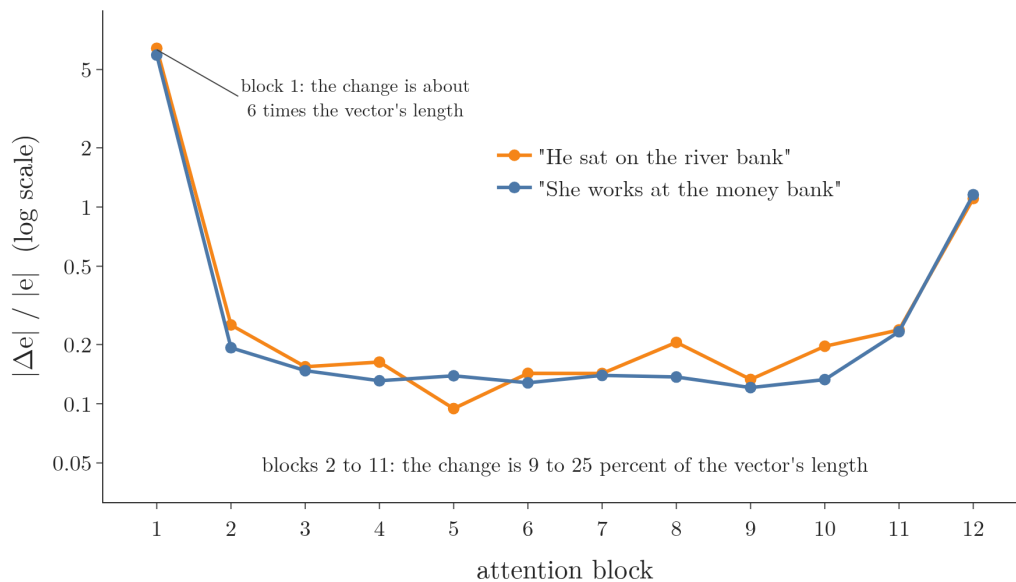


Figure 6: GPT-2 small: the length of the change added by each attention block, relative to the vector it is added to, for the last token “bank” of two real sentences (log scale)

Extra: The picture of attention as a change Δe added to a word’s vector follows Sanderson’s “Attention in transformers, step-by-step” (3Blue1Brown, 2024), recreated here with our own numbers and code.

8. Summary

Step	What happens to the arrows	“bank” in “money bank”
Projections	each embedding is moved by W_Q, W_K, W_V into three new vectors	$q_{bank} = (1.7, 1.6)$, $k_{bank} = (2.4, 0.6)$, $v_{bank} = (6.6, 3.8)$
Scores	dot product of the query with every key: how well they line up	4.45 and 5.04
Weights	divide by $\sqrt{d_k}$, softmax	0.397 on money, 0.603 on bank
Output	shrink each value vector by its weight, add tip to tail	$y_{bank} = (4.97, 4.67)$

- An embedding is an arrow; similar words point to nearby places.
- The output of self-attention is a weighted average of the value vectors, so it lies between them, pulled towards the words with large weights.
- The same word gets a different output in a different sentence: “bank” moves towards money in one and towards river in the other.
- In a transformer the attention output is a change Δe added to the word’s own vector (the residual connection); in GPT-2 small most blocks add a change of 9 to 25 percent of the vector’s length.

9. Sources

Built from

- CampusX, “Self Attention Geometric Intuition | How to Visualize Self Attention | CampusX”, YouTube, <https://www.youtube.com/watch?v=5ZgGuujZSbs>
- Sanderson, G. (3Blue1Brown), “Attention in transformers, step-by-step | Deep Learning Chapter 6”, 2024, [3blue1brown.com/lessons/attention](https://www.youtube.com/watch?v=eMlx5fFN), <https://www.youtube.com/watch?v=eMlx5fFN> 13:12–15:44 (value vectors as what to add; the change Δe added to the embedding).

Other references

- Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS 2017*. arXiv:1706.03762. Section 3.2.1, eq. 1 (scaled dot-product attention).
- Jurafsky, D. and Martin, J. H. *Speech and Language Processing*, 3rd ed. draft (19 August 2026), chapter 7: attention as a way to build contextual representations of a token’s meaning by integrating information from surrounding tokens (eq. 7.10–7.13).
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D. and Sutskever, I. (2019). Language Models are Unsupervised Multitask Learners. OpenAI. §2.3 (the GPT-2 architecture). Weights: [openai-community/gpt2](https://openai-community.github.io/gpt2) (GPT-2 small), huggingface.co.

10. Key terms

Term	Meaning
Embedding vector	The list of numbers that represents a word; drawn as an arrow from the origin
Projection	Multiplying an embedding by W_Q , W_K or W_V , which moves it to a new vector
Query, key, value vectors	The three projections of a word's embedding, used to ask, to be compared, and to be mixed
Weighted sum	The value vectors multiplied by their weights and added together
Convex hull	The smallest region with straight edges that contains a set of points; every weighted average of the points lies inside it
Contextual embedding	A word's vector after self-attention, which depends on the other words of the sentence
Change Δe	The attention output read as an update: it is added to the word's own vector by the residual connection